CSE 4309: Theory of Computing

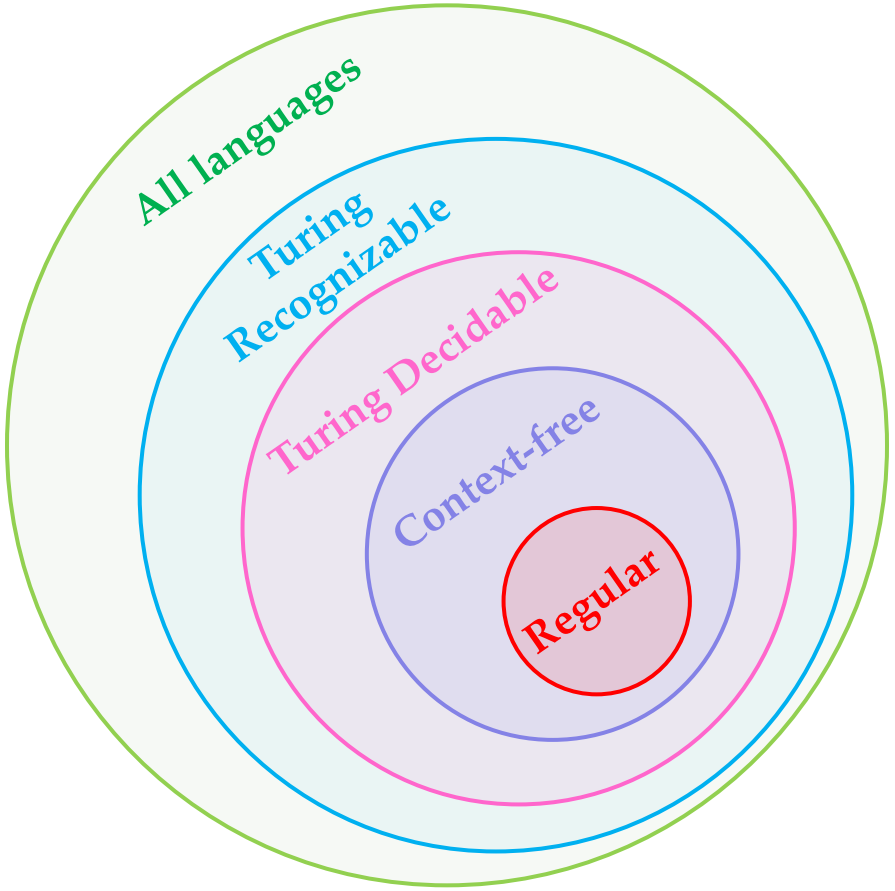
Lecture 4 — Context-Free Grammar & Pushdown Automata

Syed Rifat Raiyan
Lecturer
Department of Computer Science & Engineering
Islamic University of Technology, Dhaka, Bangladesh
Email: rifatraiyan@iut-dhaka.edu

Lecture Plan

The agenda for today

- What is Context-Free Grammar (CFG)?
- How do we create Parse Trees from CFGs?
- Ambiguity in Parse Trees
- What is a Pushdown Automaton (PDA)?
- Chomsky Normal Form (CNF) of CFGs
- CYK Algorithm
- Pumping lemma for Context-Free Languages (CFLs)



Chomsky Hierarchy

Context-Free Grammar

What is it?

In formal language theory, a context-free grammar (CFG) is a *formal grammar* whose *production rules* can be applied to a *nonterminal symbol* regardless of its *context*.

G_1

$$\left.\begin{array}{l} S \rightarrow 0S1 \\ S \rightarrow R \\ R \rightarrow \varepsilon \end{array}\right\} \text{(Substitution) Rules}$$

Rule: Variable → string of variables and terminals

Variables: Symbols appearing on left-hand side of rule

Terminals: Symbols appearing only on right-hand side

Start Variable: Top left symbol

In G_1:

3 rules

R,S

0,1

S

Example of G_1 generating a string

Tree of substitutions *"parse tree"*

S

S

Resulting string

Grammars generate strings

1. Write down start variable
2. Replace any variable according to a rule
   Repeat until only terminals remain
3. Result is the generated string
4. $L(G)$ is the language of all generated strings.

$\in L(G_1)$

$$L(G_1) = \{0^k 1^k \mid k \geq 0\}$$

Context-Free Grammar

The formal definition

Defn: A Context Free Grammar (CFG) G is a 4-tuple (V, Σ, R, S)

V finite set of variables

Σ finite set of terminal symbols

R finite set of rules **(**rule form: $V \to (V \cup \Sigma)^*$ **)**

S start variable

For $u, v \in (V \cup \Sigma)^*$ write

1) $u \Rightarrow v$ if can go from u to v with one substitution step in G

2) $u \overset{*}{\Rightarrow} v$ if can go from u to v with some number of substitution steps in G

$u \Rightarrow u_1 \Rightarrow u_2 \Rightarrow \cdots \Rightarrow u_k = v$ is called a derivation of v from u.

If $u = S$ then it is a <u>derivation</u> of v.

$L(G) = \{w \mid w \in \Sigma^* \text{ and } S \overset{*}{\Rightarrow} w\}$

Defn: A is a Context Free Language (CFL) if $A = L(G)$ for some CFG G.

Context-Free Grammar

An example

G_2

$$E \rightarrow E+T \mid T$$
$$T \rightarrow T\times F \mid F$$
$$F \rightarrow (\ E\ ) \mid a$$

Parse tree

E

E

Resulting string

$V = \{E, T, F\}$
$\Sigma = \{+, \times, (,), a\}$
$R =$ the 6 rules above
$S = E$

Generates a+a×a

Observe that the parse tree contains additional information, such as the precedence of × over + .

If a string has two different parse trees then it is derived ambiguously and we say that the grammar is ***ambiguous***.

Ambiguity

What is it?

How many reasonable distinct meanings does the following English sentence have?

"The boy saw the girl with the mirror."

Interpretation 1
The boy saw the girl who had the mirror.

Interpretation 2
The boy used the mirror to see the girl.

Ambiguity

How is this relevant here?

G_2

$E \rightarrow E+T \mid T$

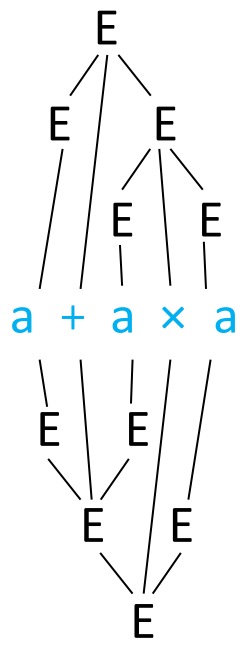
$T \rightarrow T\times F \mid F$

$F \rightarrow (\ E\ ) \mid a$

G_3

$E \rightarrow E+E \mid E\times E \mid (\ E\ ) \mid a$

Both G_2 and G_3 recognize the same language, *i.e.*, $L(G_2) = L(G_3)$.
However, G_2 is an ***unambiguous*** CFG and G_3 is ***ambiguous***.



Context-Free Grammar

Types of derivation

Leftmost Derivation: A derivation is *leftmost* if, at each step of the derivation, the leftmost variable (non-terminal) in the sentential form is the one being replaced.

Rightmost Derivation: A derivation is *rightmost* (or canonical) if, at each step of the derivation, the rightmost variable (non-terminal) in the sentential form is the one being replaced.

$$
\begin{aligned}
\mathbf{E} &\Rightarrow \mathbf{E} + T \\
&\Rightarrow \mathbf{T} + T \\
&\Rightarrow \mathbf{F} + T \\
&\Rightarrow a + \mathbf{T} \\
&\Rightarrow a + \mathbf{F} \\
&\Rightarrow a + (\mathbf{E}) \\
&\Rightarrow a + (\mathbf{T}) \\
&\Rightarrow a + (\mathbf{T} \times F) \\
&\Rightarrow a + (\mathbf{F} \times F) \\
&\Rightarrow a + (a \times \mathbf{F}) \\
&\Rightarrow a + (a \times a)
\end{aligned}
$$

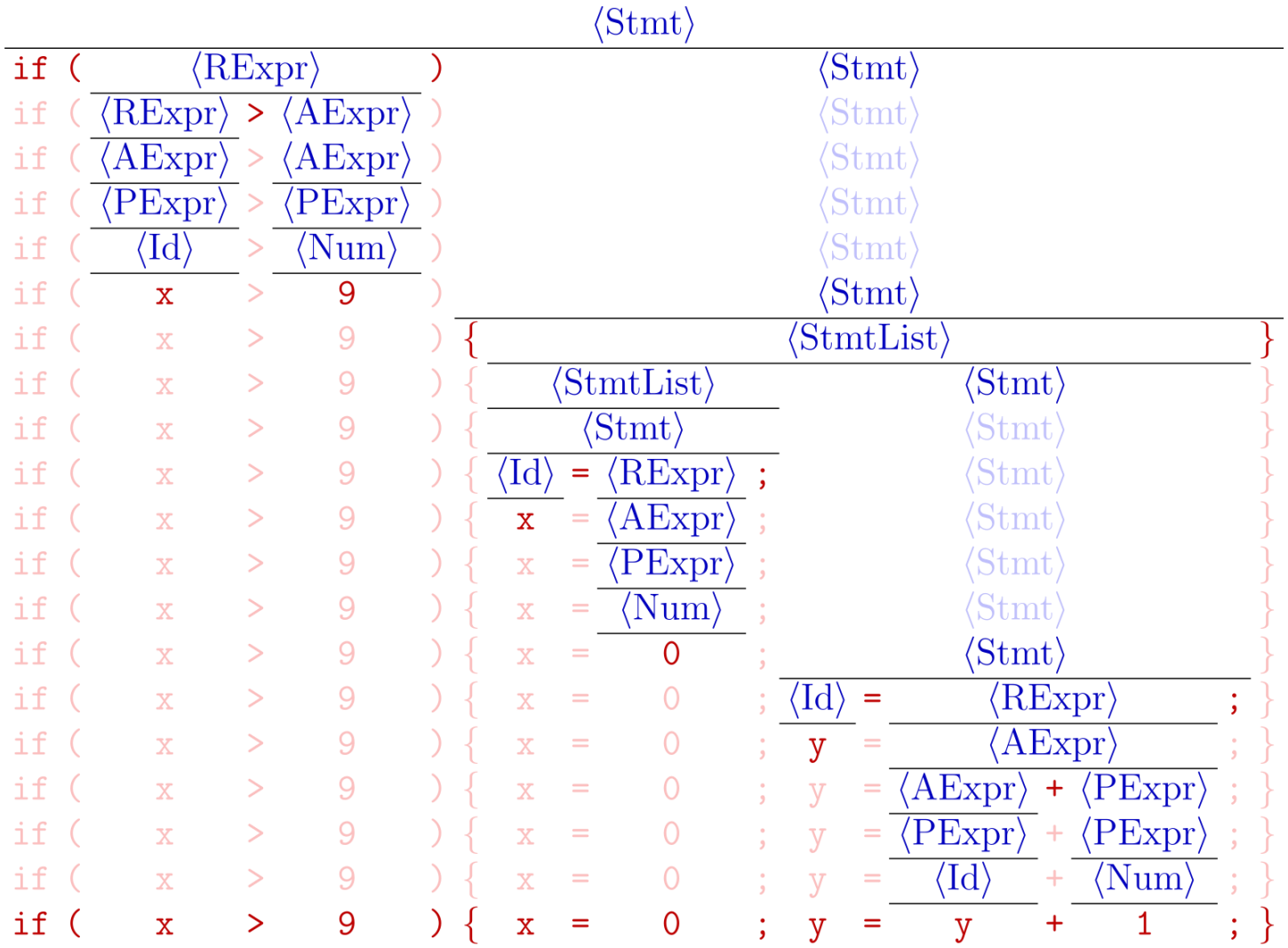
$$
\begin{aligned}
E &\rightarrow E + T \mid T \\
T &\rightarrow T \times F \mid F \\
F &\rightarrow (E) \mid a
\end{aligned}
$$

$$
\begin{aligned}
\mathbf{E} &\Rightarrow E + \mathbf{T} \\
&\Rightarrow E + \mathbf{F} \\
&\Rightarrow E + (\mathbf{E}) \\
&\Rightarrow E + (\mathbf{T}) \\
&\Rightarrow E + (\mathbf{T} \times F) \\
&\Rightarrow E + (T \times \mathbf{F}) \\
&\Rightarrow E + (\mathbf{T} \times a) \\
&\Rightarrow E + (\mathbf{F} \times a) \\
&\Rightarrow \mathbf{E} + (a \times a) \\
&\Rightarrow \mathbf{T} + (a \times a) \\
&\Rightarrow \mathbf{F} + (a \times a) \\
&\Rightarrow a + (a \times a)
\end{aligned}
$$

Context-Free Grammar

An application

$$
\begin{aligned}
\langle \text{Stmt} \rangle &\to \langle \text{Id} \rangle \texttt{ = } \langle \text{RExpr} \rangle \texttt{ ;} \\
\langle \text{Stmt} \rangle &\to \texttt{\{ } \langle \text{StmtList} \rangle \texttt{ \}} \\
\langle \text{Stmt} \rangle &\to \texttt{if (} \langle \text{RExpr} \rangle \texttt{)} \ \langle \text{Stmt} \rangle \\
\langle \text{StmtList} \rangle &\to \langle \text{Stmt} \rangle \\
\langle \text{StmtList} \rangle &\to \langle \text{StmtList} \rangle \ \langle \text{Stmt} \rangle \\
\langle \text{RExpr} \rangle &\to \langle \text{RExpr} \rangle \texttt{ > } \langle \text{AExpr} \rangle \\
\langle \text{RExpr} \rangle &\to \langle \text{RExpr} \rangle \texttt{ < } \langle \text{AExpr} \rangle \\
\langle \text{RExpr} \rangle &\to \langle \text{RExpr} \rangle \texttt{ >= } \langle \text{AExpr} \rangle \\
\langle \text{RExpr} \rangle &\to \langle \text{RExpr} \rangle \texttt{ <= } \langle \text{AExpr} \rangle \\
\langle \text{RExpr} \rangle &\to \langle \text{AExpr} \rangle \\
\langle \text{AExpr} \rangle &\to \langle \text{AExpr} \rangle \texttt{ + } \langle \text{PExpr} \rangle \\
\langle \text{AExpr} \rangle &\to \langle \text{AExpr} \rangle \texttt{ - } \langle \text{PExpr} \rangle \\
\langle \text{AExpr} \rangle &\to \langle \text{PExpr} \rangle \\
\langle \text{PExpr} \rangle &\to \langle \text{Id} \rangle \\
\langle \text{PExpr} \rangle &\to \langle \text{Num} \rangle \\
\langle \text{Id} \rangle &\to \texttt{x} \\
\langle \text{Id} \rangle &\to \texttt{y} \\
\langle \text{Num} \rangle &\to \texttt{0} \\
\langle \text{Num} \rangle &\to \texttt{1} \\
\langle \text{Num} \rangle &\to \texttt{9}
\end{aligned}
$$



Simplified excerpt of the formal grammar for the C programming language

Context-Free Grammar

Some examples

$L = \{w \mid w \text{ has properly nested parentheses}\}$

$$S \rightarrow (S) \mid SS \mid \varepsilon$$

$L = \{\mathrm{a}^n \mathrm{b}^m \mid n, m \geq 1\}$

$$S \rightarrow AB$$
$$A \rightarrow aA \mid a$$
$$B \rightarrow bB \mid b$$

What if $n, m \geq 0$?

$L = \{\mathrm{a}^n \mathrm{b}^n \mathrm{c}^m \mid n, m \geq 1\}$

$$S \rightarrow AC$$
$$A \rightarrow aAb \mid ab$$
$$C \rightarrow cC \mid c$$

$L = \{\mathrm{a}^n \mathrm{b}^{2n} \mid n \geq 1\}$

$$S \rightarrow aSbb \mid abb$$

Context-Free Grammar

Some more examples

$L = \{w \mid w \text{ has no. of } 0'\text{s} = \text{no. of } 1'\text{s}\}$

$$
\begin{aligned}
S &\rightarrow 0S1 \mid 1S0 \\
S &\rightarrow 01S \mid 10S \\
S &\rightarrow S01 \mid S10 \\
S &\rightarrow SS \\
S &\rightarrow \varepsilon
\end{aligned}
$$

$$S \rightarrow 0S1 \mid 1S0 \mid SS \mid \varepsilon$$

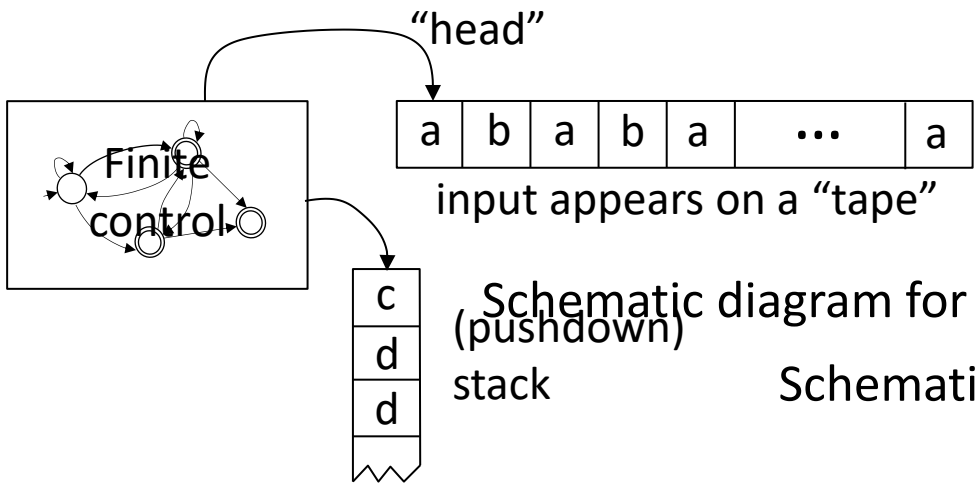
Less redundant!

$L = \{w \in \{\text{a}, \text{b}\}* \mid w \text{ is a palindrome}\}$

$$S \rightarrow aSa \mid bSb \mid a \mid b \mid \varepsilon$$

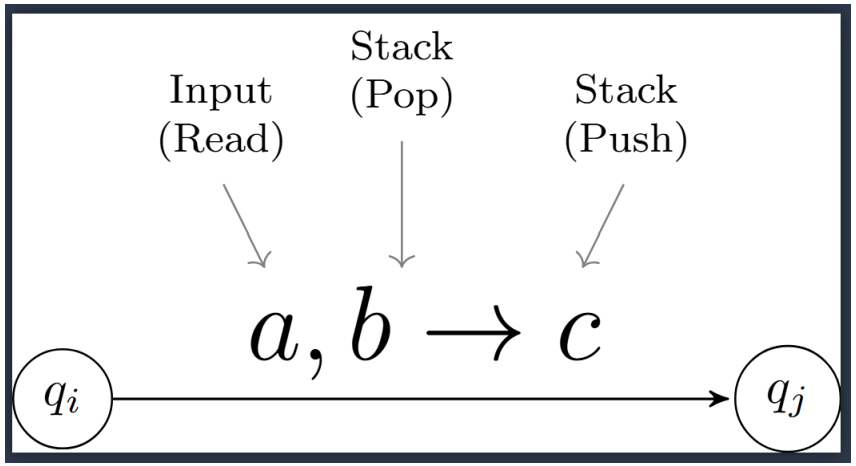
Pushdown Automaton (PDA)

What is it?



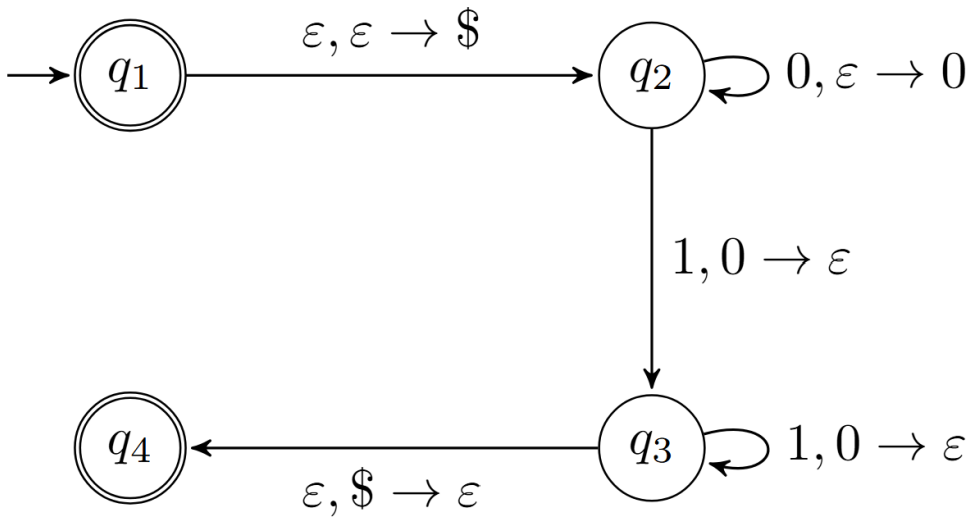
Schematic diagram for DFA or NFA

Schematic diagram for PDA



Operates like an NFA except can <u>write-add</u> or <u>read-remove</u> symbols from the top of stack.

push pop



Example: PDA for $D = \{0^k 1^k \mid k \geq 0\}$

1) Read 0s from input, push onto stack until read 1.
2) Read 1s from input, while popping 0s from stack.
3) Enter accept state if stack is empty. (note: acceptance only at end of input)

Pushdown Automaton (PDA)

The formal definition

Defn: A <u>Pushdown Automaton</u> (PDA) is a 6-tuple $(Q, \Sigma, \Gamma, \delta, q_0, F)$

Σ input alphabet

Γ stack alphabet

δ: $\mathrm{Q} \times \Sigma_\varepsilon \times \Gamma_\varepsilon \to \mathcal{P}(Q \times \Gamma_\varepsilon)$

$\delta(q, \mathrm{a}, \mathrm{c}) = \{(r_1, \mathrm{d}),\ (r_2, \mathrm{e})\}$

Accept if some thread is in the accept state at the end of the input string.

Example: PDA for $B = \{ww^{\mathcal{R}} \mid w \in \{0,1\}^*\}$ Sample input:

0	1	1	1	1	0

1) Read and push input symbols.
   Nondeterministically either repeat or go to (2).
2) Read input symbols and pop stack symbols, compare.
   If ever $\neq$ then thread rejects.
3) Enter accept state if stack is empty. (do in "software")

The nondeterministic forks replicate the stack.

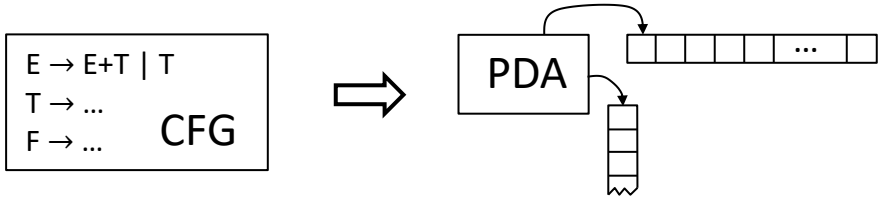
This language requires nondeterminism.
Our PDA model is nondeterministic.

Pushdown Automaton (PDA)

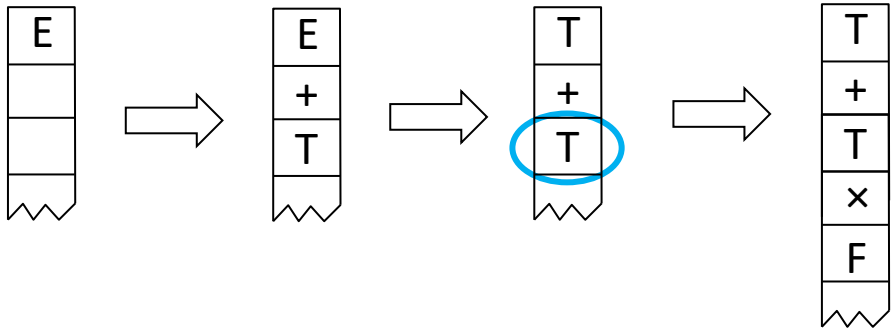
Equivalence of CFG and PDA

Theorem: If A is a CFL then some PDA recognizes A

Proof: Convert A's CFG to a PDA



IDEA: PDA begins with starting variable and guesses substitutions.
It keeps intermediate generated strings on stack. When done, compare with input.



Input:

a	+	a	×	a

G_2

$$E \rightarrow E+T \mid T$$
$$T \rightarrow T\times F \mid F$$
$$F \rightarrow (\,E\,) \mid a$$

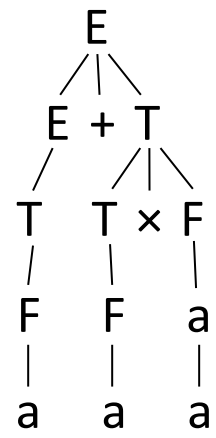
E

E+T

T+T×F

F+F×a

a+a×a



Problem! Access below the top of stack is cheating!

Instead, only substitute variables when on the top of stack.

If a terminal is on the top of stack, pop it and compare with input. Reject if ≠.

Pushdown Automaton (PDA)

Equivalence of CFG and PDA

G_2

$E \rightarrow E+T \mid T$

$T \rightarrow T\times F \mid F$

$F \rightarrow (\,E\,) \mid a$

Theorem: If A is a CFL then some PDA recognizes A

Proof construction: Convert the CFG for A to the following PDA.

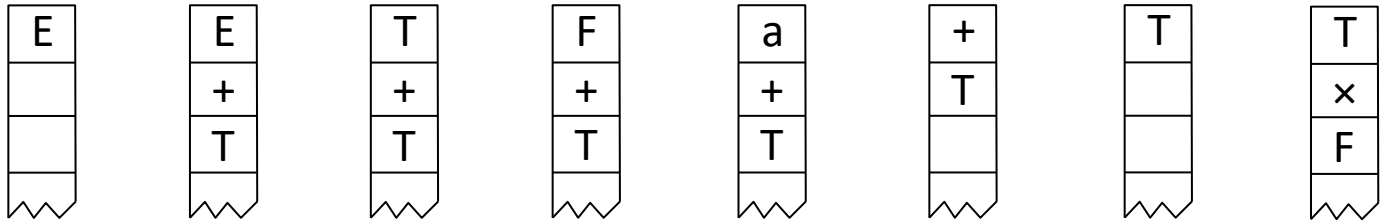
1) Push the start symbol on the stack.
2) If the top of stack is

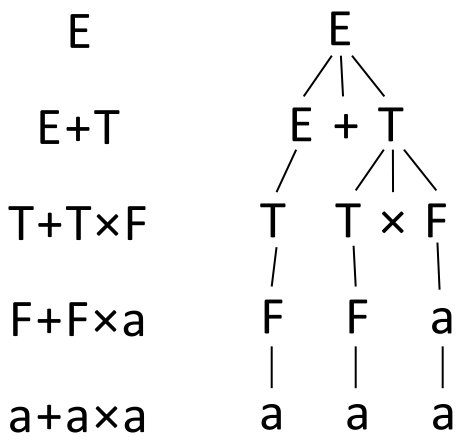
   Variable: replace with right hand side of rule (nondeterministic choice).

   Terminal: pop it and match with next input symbol.
3) If the stack is empty, *accept*.

Example:

a	+	a	×	a





Pushdown Automaton (PDA)

Equivalence of CFG and PDA

Theorem: A is a CFL iff* some PDA recognizes A

$\longleftrightarrow$ Done.

In book (lemma 2.27 in 3rd edition).

You are responsible for knowing it is true, but not for knowing the proof.

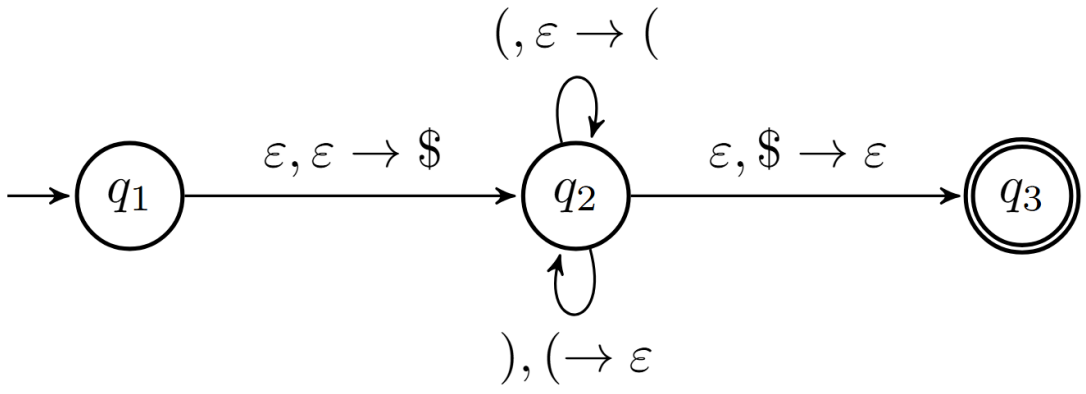
* "iff" = "if an only if" means the implication goes both ways.

So we need to prove both directions: forward (→) and reverse (←).

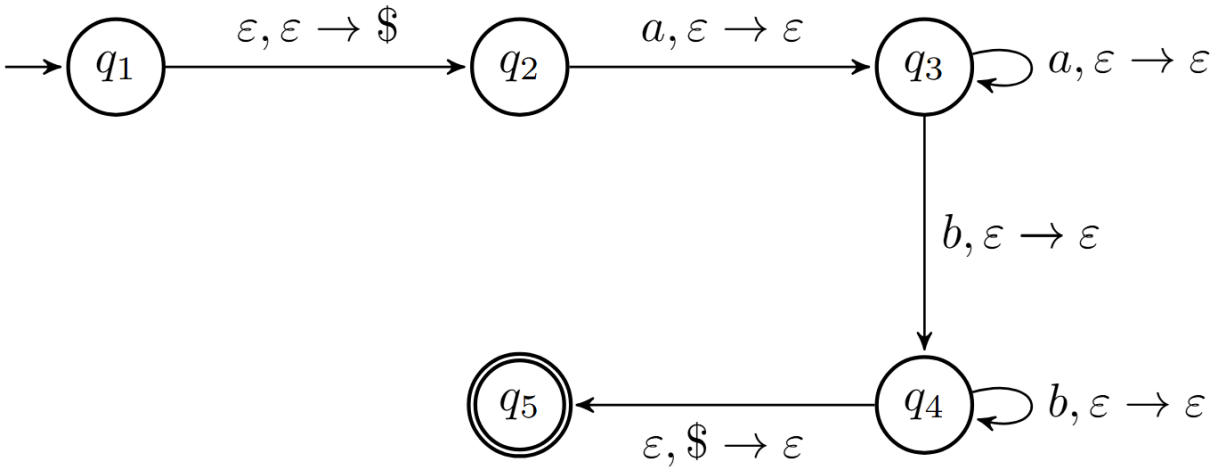
Pushdown Automaton (PDA)

Some examples

$L = \{w \mid w \text{ has properly nested parentheses}\}$



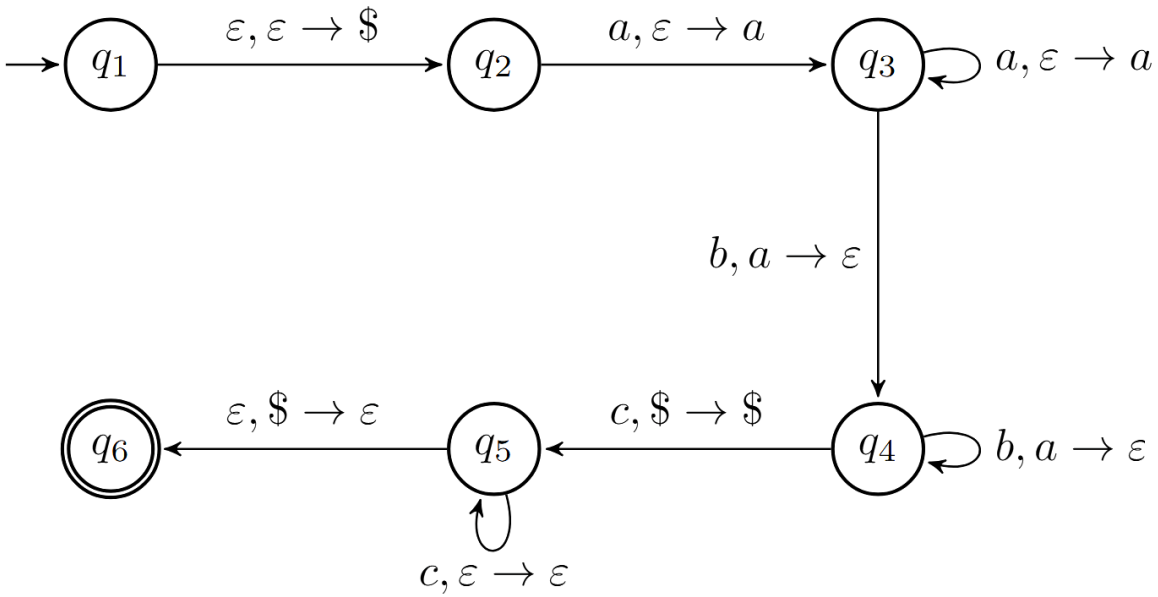
$L = \{a^n b^m \mid n, m \geq 1\}$



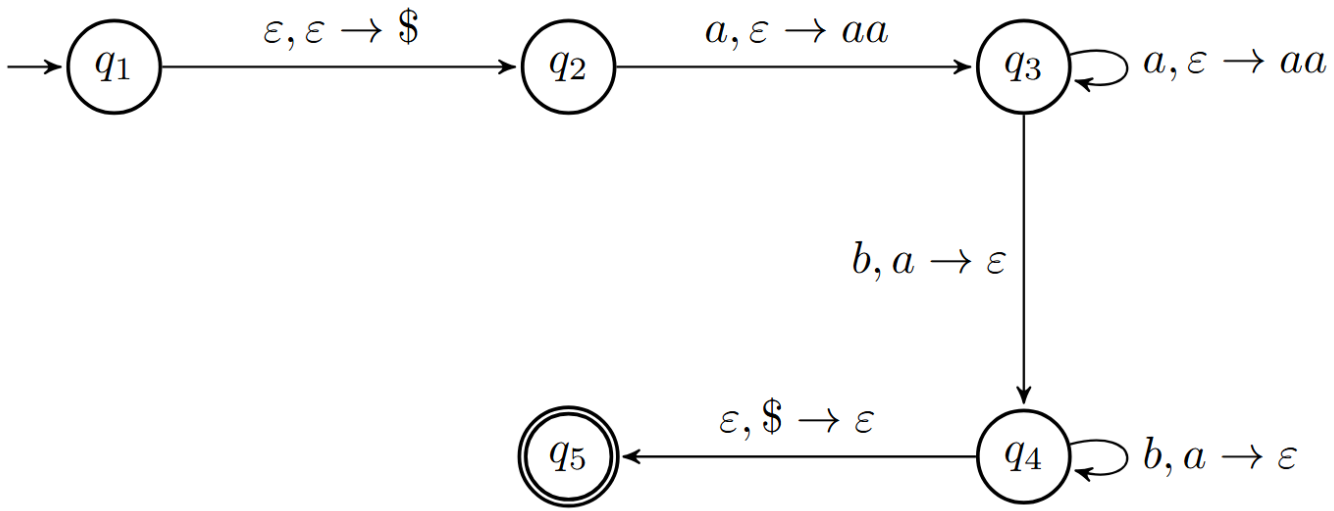
Pushdown Automaton (PDA)

Some examples

$$L = \{a^n b^n c^m \mid n, m \geq 1\}$$



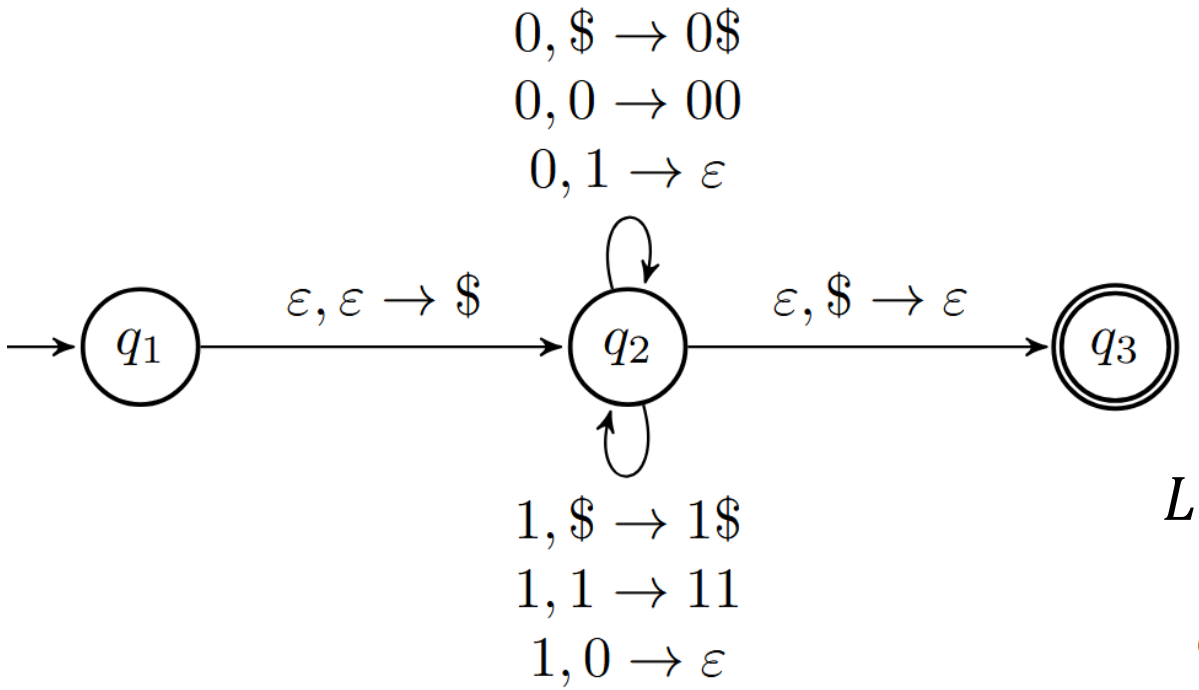
$$L = \{a^n b^{2n} \mid n \geq 1\}$$



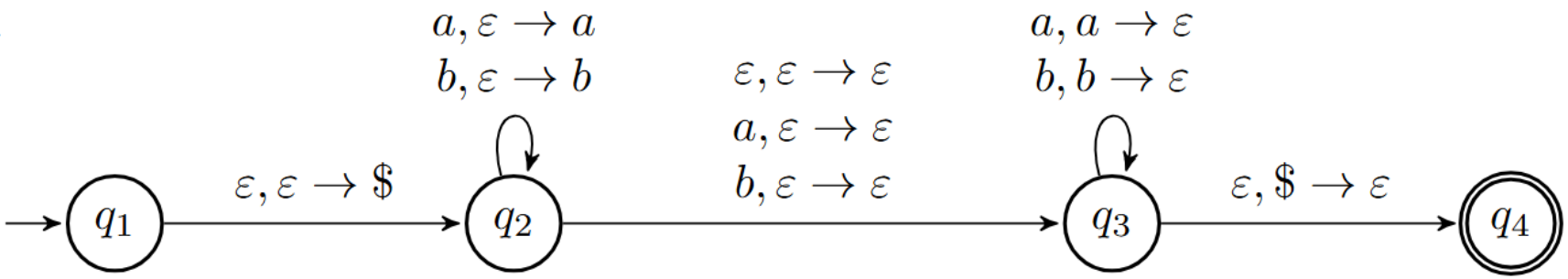
Pushdown Automaton (PDA)

Some examples

$$L = \{w \mid w \text{ has no. of } 0's = \text{no. of } 1's\}$$



$$L = \{w \in \{a, b\}* \mid w \text{ is a palindrome}\}$$



Chomsky Normal Form (CNF)

What is it?

A context-free grammar is in ***Chomsky normal form*** if every rule is of the form

$$A \to BC$$
$$A \to a$$

where a is any terminal and A, B, and C are any variables—except that B and C may not be the start variable. In addition, we permit the rule $S \to \varepsilon$, where S is the start variable.

Theorem: Any CFL is generated by a CFG in Chomsky normal form.

Chomsky Normal Form (CNF)

Steps of conversion

1. **New Start Symbol**: Add a new start variable S_0 and the rule $S_0 \to S$.
   - Ensures the start variable never appears on the right-hand side.
2. **Eliminate ε-rules**: Remove any rule $A \to \varepsilon$ (where $A \neq S_0$).
   - For each rule $R \to uAv$, add a new rule $R \to uv$.
   - If a rule has multiple occurrences of A, add rules for all combinations of deletions.
3. **Eliminate Unit Rules**: Remove any rule $A \to B$.
   - Whenever $B \to u$ exists, add the rule $A \to u$.
   - Repeat until all rules of the form $Variable \to Variable$ are removed.
4. **Convert Remaining Rules**:
   - **Binarization**: Replace long rules $A \to u_1 u_2 \ldots u_k$ $(k \geq 3)$ with a chain of rules: $A \to u_1 A_1, A_1 \to u_2 A_2, A_2 \to u_3 A_3, \ldots, A_{k-2} \to u_{k-1} u_k$.
   - **Terminal Replacement**: Replace any terminal a in a multi-symbol rule with a new variable U_a and add the rule $U_a \to a$.

Order must be maintained

Chomsky Normal Form (CNF)

An example

$$S \to ASA \mid \mathtt{a}B$$
$$A \to B \mid S$$
$$B \to \mathtt{b} \mid \varepsilon$$

Step-1: Make a new start variable.

$$\boldsymbol{S_0 \to S}$$
$$S \to ASA \mid \mathtt{a}B$$
$$A \to B \mid S$$
$$B \to \mathtt{b} \mid \varepsilon$$

Step-2: Remove ε–rules B → ε and A → ε.

$$S_0 \to S$$
$$S \to ASA \mid \mathtt{a}B \mid \mathbf{a}$$
$$A \to B \mid S \mid \boldsymbol{\varepsilon}$$
$$B \to \mathtt{b} \mid \varepsilon$$

$$S_0 \to S$$
$$S \to ASA \mid \mathtt{a}B \mid \mathtt{a} \mid \boldsymbol{SA} \mid \boldsymbol{AS} \mid \boldsymbol{S}$$
$$A \to B \mid S \mid \varepsilon$$
$$B \to \mathtt{b}$$

Step-3: Remove unit rules S → S and S_0 → S.

$$S_0 \to S$$
$$S \to ASA \mid \mathtt{a}B \mid \mathtt{a} \mid SA \mid AS \mid S$$
$$A \to B \mid S$$
$$B \to \mathtt{b}$$

$$S_0 \to S \mid \boldsymbol{ASA} \mid \mathbf{a}\boldsymbol{B} \mid \mathbf{a} \mid \boldsymbol{SA} \mid \boldsymbol{AS}$$
$$S \to ASA \mid \mathtt{a}B \mid \mathtt{a} \mid SA \mid AS$$
$$A \to B \mid S$$
$$B \to \mathtt{b}$$

Step-4: Remove unit rules A → B and A → S.

$$S_0 \to ASA \mid \mathtt{a}B \mid \mathtt{a} \mid SA \mid AS$$
$$S \to ASA \mid \mathtt{a}B \mid \mathtt{a} \mid SA \mid AS$$
$$A \to B \mid S \mid \mathbf{b}$$
$$B \to \mathtt{b}$$

$$S_0 \to ASA \mid \mathtt{a}B \mid \mathtt{a} \mid SA \mid AS$$
$$S \to ASA \mid \mathtt{a}B \mid \mathtt{a} \mid SA \mid AS$$
$$A \to S \mid \mathtt{b} \mid \boldsymbol{ASA} \mid \mathbf{a}\boldsymbol{B} \mid \mathbf{a} \mid \boldsymbol{SA} \mid \boldsymbol{AS}$$
$$B \to \mathtt{b}$$

Step-5: Convert the remaining rules into the proper form by adding new variables and rules.

$$S_0 \to AA_1 \mid UB \mid \mathtt{a} \mid SA \mid AS$$
$$S \to AA_1 \mid UB \mid \mathtt{a} \mid SA \mid AS$$
$$A \to \mathtt{b} \mid AA_1 \mid UB \mid \mathtt{a} \mid SA \mid AS$$
$$A_1 \to SA$$
$$U \to \mathtt{a}$$
$$B \to \mathtt{b}$$

Cocke–Younger–Kasami (CYK) algorithm

What is it?

Purpose:

A membership algorithm to determine if a string w belongs to the language $L(G)$ of a Context-Free Grammar.

Prerequisite: The grammar G must be in **Chomsky Normal Form (CNF)**, meaning all rules are of the form:

$$A \to BC \quad \text{or} \quad A \to a$$

Let $w = s_1 s_2 \ldots s_n$ be a string of length n. We construct a triangular table T where $T[i, j]$ contains the set of variables that generate the substring of w starting at index i with length j.

1. **Base Case (Substrings of length 1):** For each position i from 1 to n:

$$T[i, 1] = \{A \mid A \to s_i \in R\}$$

2. **Recursive Step (Substrings of length $j > 1$):** To find $T[i, j]$, we split the substring into two parts: a prefix of length k and a suffix of length $j - k$.

$$T[i, j] = \{A \mid A \to BC \in R, \text{ where } B \in T[i, k] \text{ and } C \in T[i + k, j - k]\}$$

   This is calculated for all possible split points k (where $1 \leq k < j$).

3. **Acceptance:** The string w is accepted if the start variable S is in the top cell of the table:

$$S \in T[1, n]$$

CYK algorithm

An example

$S \to AB \mid BC$

$A \to BA \mid a$

$B \to CC \mid b$

$C \to AB \mid a$

Target String: $w = baaba$

Level 1 (Length $j = 1$):

- $T_{1,1}$: $\{B\}$ (from $B \to b$)
- $T_{2,1}$: $\{A, C\}$ (from $A \to a, C \to a$)
- $T_{3,1}$: $\{A, C\}$ (from $A \to a, C \to a$)
- $T_{4,1}$: $\{B\}$ (from $B \to b$)
- $T_{5,1}$: $\{A, C\}$ (from $A \to a, C \to a$)

Level 2 (Length $j = 2$):

- $T_{1,2} = T_{1,1}T_{2,1} = \{B\}\{A, C\} = \{BA, BC\} \implies \{A, S\}$
- $T_{2,2} = T_{2,1}T_{3,1} = \{A, C\}\{A, C\} = \{AA, AC, CA, CC\} \implies \{B\}$
- $T_{3,2} = T_{3,1}T_{4,1} = \{A, C\}\{B\} = \{AB, CB\} \implies \{S, C\}$
- $T_{4,2} = T_{4,1}T_{5,1} = \{B\}\{A, C\} = \{BA, BC\} \implies \{A, S\}$

Level 3 (Length $j = 3$):

- $T_{1,3} = (T_{1,1}T_{2,2}) \cup (T_{1,2}T_{3,1}) = \{B\}\{B\} \cup \{S, A\}\{A, C\} = \{BB, SA, SC, AA, AC\} \implies \emptyset$
- $T_{2,3} = (T_{2,1}T_{3,2}) \cup (T_{2,2}T_{4,1}) = \{A, C\}\{S, C\} \cup \{B\}\{B\} = \{AS, AC, CS, CC, BB\} \implies \{B\}$
- $T_{3,3} = (T_{3,1}T_{4,2}) \cup (T_{3,2}T_{5,1}) = \{A, C\}\{S, A\} \cup \{S, C\}\{A, C\} = \{AS, AA, CS, CA, SA, SC, CA, CC\} \implies \{B\}$

Level 4 (Length $j = 4$):

- $T_{1,4} = (T_{1,1}T_{2,3}) \cup (T_{1,2}T_{3,2}) \cup (T_{1,3}T_{4,1}) = \{B\}\{B\} \cup \{S, A\}\{S, C\} \cup \emptyset = \emptyset$
- $T_{2,4} = (T_{2,1}T_{3,3}) \cup (T_{2,2}T_{4,2}) \cup (T_{2,3}T_{5,1}) = \{A, C\}\{B\} \cup \{B\}\{S, A\} \cup \{B\}\{A, C\} \implies \{S, C, A\}$

Level 5 (Length $j = 5$):

- $T_{1,5} = (T_{1,1}T_{2,4}) \cup (T_{1,2}T_{3,3}) \cup (T_{1,3}T_{4,2}) \cup (T_{1,4}T_{5,1})$

  $= \{B\}\{S, C, A\} \cup \{S, A\}\{B\} \cup \emptyset \cup \emptyset$

  $= \{BS, BC, BA, SB, AB\} \implies \{S, A, C\}$

CYK algorithm

An example

$S \to AB \mid BC$

$A \to BA \mid a$

$B \to CC \mid b$

$C \to AB \mid a$

Target String: $w = baaba$

The table $T_{i,j}$ represents the set of variables that generate the substring of w starting at index i with length j.

Length (j)	**Starting Position (i)**				
$j = 5$	$\{S, A, C\}$				
$j = 4$	$\emptyset$	$\{S, A, C\}$			
$j = 3$	$\emptyset$	$\{B\}$	$\{B\}$		
$j = 2$	$\{S, A\}$	$\{B\}$	$\{S, C\}$	$\{S, A\}$	
$j = 1$	$\{B\}$	$\{A, C\}$	$\{A, C\}$	$\{B\}$	$\{A, C\}$
Input:	b	a	a	b	a
Index (i):	1	2	3	4	5

Since the start variable S is in the top-most cell $T_{1,5}$, the string $w = baaba$ is **accepted** by the grammar ($w \in L(G)$).

Mini Quiz

Time and Space Complexity

What is the *time* complexity of the CYK algorithm?
What about the *space* complexity?

Time Complexity: $O(n^3 \cdot |G|)$, where n is the length of the string and $|G|$ is the number of rules in the grammar.

Space Complexity: $O(n^2)$ to store the triangular table.

Pumping Lemma

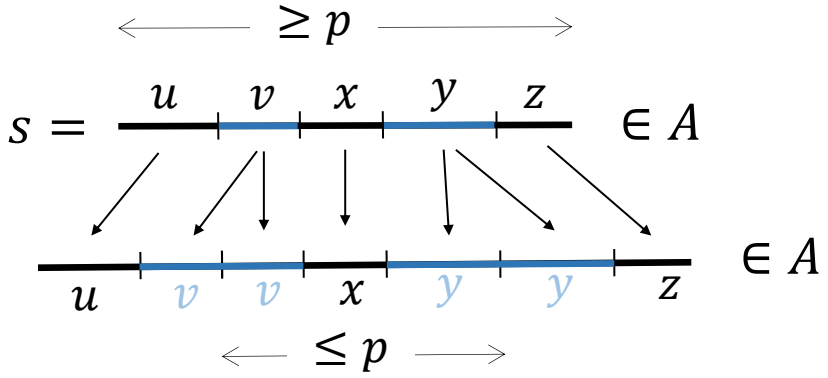
For Non-Context-Free Languages

Let $B = \{0^k 1^k 2^k \mid k \geq 0\}$. We will show that B isn't a CFL.

Pumping Lemma for CFLs: For every CFL A, there is a p such that if $s \in A$ and $|s| \geq p$ then $s = uvxyz$ where

1) $uv^i xy^i z \in A$ for all $i \geq 0$
2) $vy \neq \varepsilon$
3) $|vxy| \leq p$

Informally: All long strings in A are pumpable and stay in A.



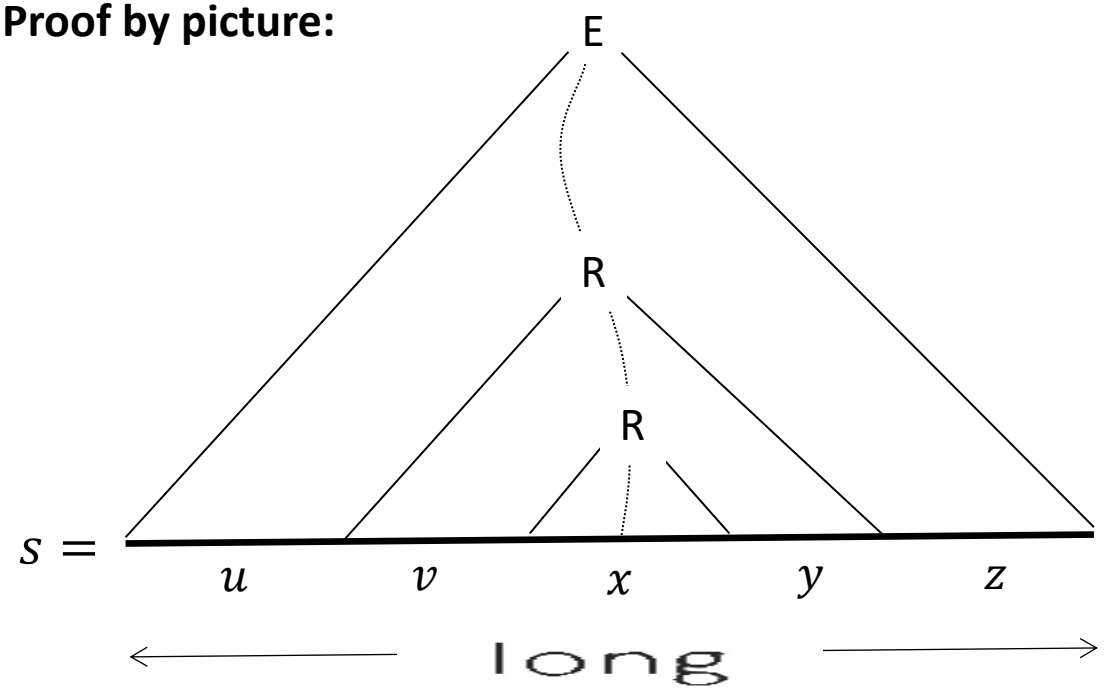
Pumping Lemma

Visual Proof

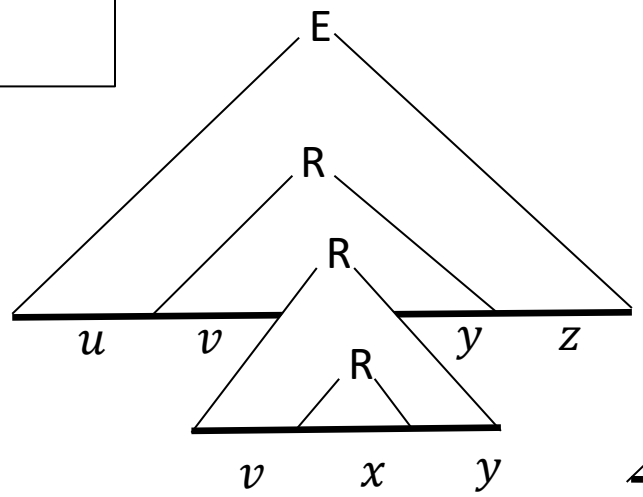
Pumping Lemma for CFLs: For every CFL A, there is a p such that if $s \in A$ and $|s| \geq p$ then $s = uvxyz$ where

1) $uv^ixy^iz \in A$ for all $i \geq 0$
2) $vy \neq \varepsilon$
3) $|vxy| \leq p$

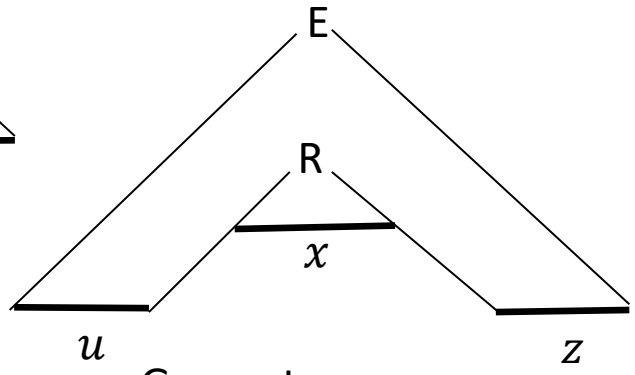
Proof by picture:



Long $s \rightarrow$
tall parse tree



Generates $uvvxyyz$
$= uv^2xy^2z$



Generates uxz
$= uv^0xy^0z$

"cutting and pasting" argument

Pumping Lemma

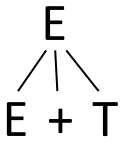
Detailed Proof

For $s \in A$ where $|s| \geq p$, we have $s = uvxyz$ where:

1) $uv^ixy^iz \in A$ for all $i \geq 0$
2) $vy \neq \varepsilon$
3) $|vxy| \leq p$

Let b = the length of the longest right hand side of a rule (E → E+T)

= the max branching of the parse tree



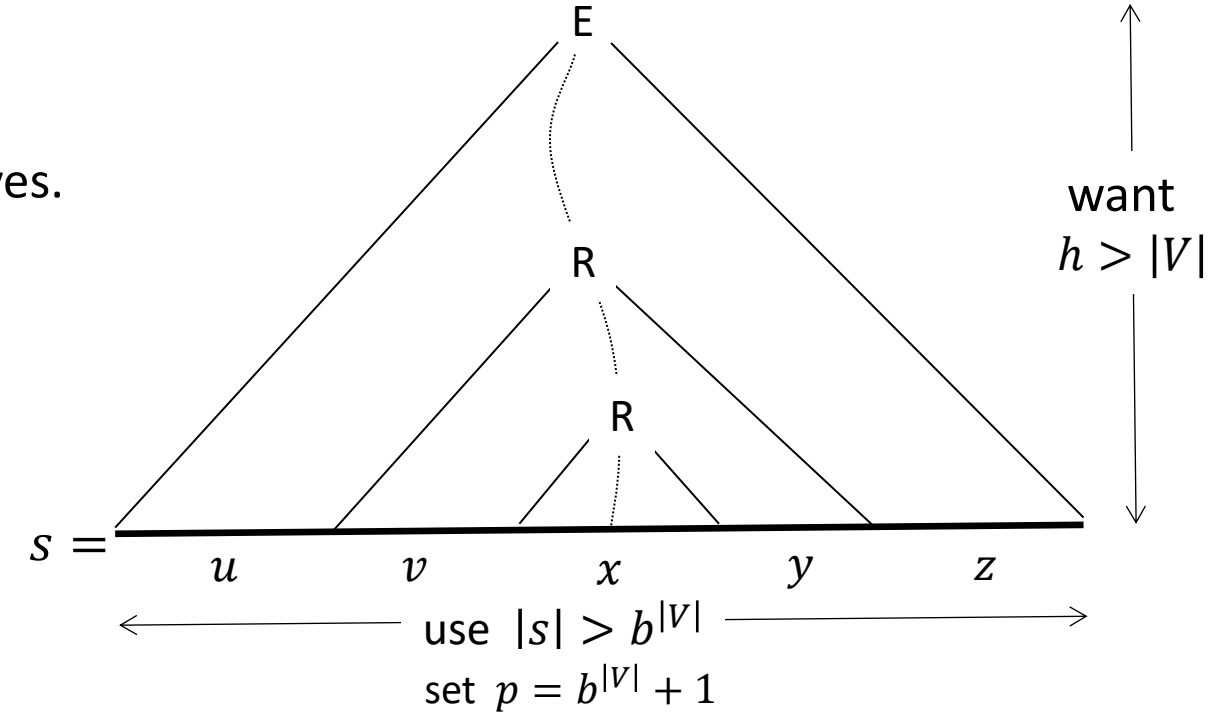
Let h = the height of the parse tree for s.

A tree of height h and max branching b has at most b^h leaves.
So $|s| \leq b^h$.

Let $p = b^{|V|} + 1$ where $|V|$ = # variables in the grammar.

So if $|s| \geq p > b^{|V|}$ then $|s| > b^{|V|}$ and so $h > |V|$.

Thus at least $|V| + 1$ variables occur in the longest path.
So some variable R must repeat on a path.



Pumping Lemma

An example

Pumping Lemma for CFLs: For every CFL A, there is a p such that if $s \in A$ and $|s| \geq p$ then $s = uvxyz$ where

1) $uv^ixy^iz \in A$ for all $i \geq 0$
2) $vy \neq \varepsilon$
3) $|vxy| \leq p$

Let $B = \{0^k 1^k 2^k \mid k \geq 0\}$

Show: B is not a CFL

Proof by Contradiction:

Assume (to get a contradiction) that B <u>is</u> a CFL .

The CFL pumping lemma gives p as above. Let $s = 0^p 1^p 2^p \in B$.

Pumping lemma says that can divide $s = uvxyz$ satisfying the 3 conditions.

Condition 3 ($|vxy| \leq p$) implies that vxy cannot contain both 0s and 2s.

So uv^2xy^2z has unequal numbers of 0s, 1s, and 2s.

Thus $uv^2xy^2z \notin B$, violating Condition 1. Contradiction!

Therefore our assumption (B is a CFL) is false. We conclude that B is not a CFL .

$$s = \underbrace{00\cdots0011\cdots1122\cdots22}_{u \mid v \mid x \mid y \mid z}$$

$\leftarrow \leq p \rightarrow$

Pumping Lemma

An example

Pumping Lemma for CFLs: For every CFL A, there is a p such that if $s \in A$ and $|s| \geq p$ then $s = uvxyz$ where

1) $uv^ixy^iz \in A$ for all $i \geq 0$
2) $vy \neq \varepsilon$
3) $|vxy| \leq p$

Let $F = \{ww \mid w \in \Sigma^*\}$. $\Sigma = \{0,1\}$.

Show: F is not a CFL.

$$s_1 = \underbrace{000 \cdots 001000 \cdots 001}_{u \quad \overbrace{v\ x\ y}^{\leq p} \quad z}$$

Assume (for contradiction) that F is a CFL.
The CFL pumping lemma gives p as above. Need to choose $s \in F$. Which s?

Try $s_1 = 0^p 1 0^p 1 \in F$.

Try $s_2 = 0^p 1^p 0^p 1^p \in F$.

$$s_2 = \underbrace{0 \cdots 01 \cdots 10 \cdots 01 \cdots 1}_{u \quad \overbrace{v\ x\ y}^{\leq p} \quad z}$$

Show s_2 cannot be pumped $s_2 = uvxyz$ satisfying the 3 conditions.
Condition 3 implies that vxy does not overlap two runs of 0s or two runs of 1s.
Therefore, in uv^2xy^2z, two runs of 0s or two runs of 1s have unequal length.
So $uv^2xy^2z \notin F$ violating Condition 1. Contradiction! Thus, F is not a CFL.